Phase6 Configuration Qa

Moderated Learning Management System (LMS)

Phase 6 — Configuration Management and Quality Assurance

6.1 Repository and branching policy

Use Git with main as the protected, releasable branch. Create short-lived branches named feature/<ticket>-<description>, fix/<ticket>-<description>, docs/<topic>, or test/<topic>. Push the branch and open a pull request. A pull request requires a successful CI build/test run and one reviewer. Squash or merge with a meaningful message. Never commit passwords, database files, submission files, build directories, or private learner data.

Suggested workflow:

```
git switch -c feature/CR1-class-extensions
git add include src tests docs CMakeLists.txt .github
 git commit -m "feat: add lecturer class-wide deadline extensions"
git push -u origin feature/CR1-class-extensions
```

After review and merge:

```
git switch main
git pull origin main
git tag -a v1.1 -m "Moderated LMS v1.1 - class-wide extensions"
git push origin v1.1
```

Use annotated tags for releases. v1.0 identifies the Phase 4 baseline; v1.1 contains CR1.

6.2 CI pipeline

.github/workflows/build.yml runs on every push and pull request. It installs CMake, Ninja, GCC, OpenSSL, cppcheck and gcovr; configures CMake; builds; runs CTest; runs cppcheck; and publishes a coverage report as a workflow artifact. The GoogleTest suite is enabled in CI and is fetched by CMake. CI must be green before merging.

6.3 Static analysis

The project enables -Wall -Wextra -Wpedantic. CI additionally runs cppcheck --enable=warning,performance,portability over src and include. GCC's -fanalyzer can be used as an additional compiler analysis pass. The report should be regenerated from the exact release commit; third-party SQLite amalgamation is excluded from application static-analysis scope.

6.4 Code metrics

The current application source baseline is 561 lines in src/lms.cpp, 215 lines in include/lms.hpp, 11 lines in src/main.cpp, and 63 lines in the local CTest harness (850 total across these four files; generated SQLite source is excluded). This is a simple physical LOC count, not a measure of quality.

Cyclomatic complexity is estimated as 1 plus the number of independent decision points (if, else if, for, while, case, catch, and conditional operators) in a function. This is a lightweight

review estimate, not a certified metric; use a dedicated analyser such as Lizard for a formal report. Priority for refactoring is the console runApplication dispatcher and service operations with several permission/validation branches.

6.5 Self-review checklist and record

Checklist item	Review result / action
SQL values are bound through prepared statements	Existing repository Statement wrapper binds values; continue review for every new query
No plaintext passwords	PBKDF2-HMAC-SHA-256 with random salt via OpenSSL; verified by tests
Role checks in service layer	CR1 method checks active lecturer and assigned course; negative test added
Exceptions handled at UI boundary	Interactive loop catches std::exception; invalid operations return user-facing messages
Data access isolated	SQL remains in repository/database layer; CR1 query added to AssignmentRepository
Boundary cases tested	Deadline, late-day, password length, extension reason and permission cases covered
Audit trail	Existing mark audit records actor/time/old/new/reason; verify moderation and repeat-marking paths with further tests
Accessibility	Console labels are text-based; future GUI must provide keyboard navigation, screen-reader labels and sufficient contrast
Secrets and generated files excluded	.gitignore excludes build/database/storage artifacts; verify before each push
Documentation current	Phase 5–8 docs and CR1 diagram updated in this release

Review changes made: class-wide extension table and service gate added; empty-reason validation added; student deadline display updated; GoogleTest suite and CI job added; coverage gap explicitly reported rather than claiming 70% without evidence.

References

The following sources support the standards, tools and practices discussed in this document. URLs were checked on 2 November 2026.

1. Chacon, S., & Straub, B. (n.d.). Git Book: Branching Workflows. Pro Git. https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows
2. GitHub. (n.d.). Building and testing CMake projects. GitHub Actions documentation. https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-cmake
3. Kitware. (n.d.). CMake documentation. https://cmake.org/cmake/help/latest/
4. Cppcheck. (n.d.). Cppcheck manual. https://cppcheck.sourceforge.io/manual.html
5. ISO/IEC. (2017). Programming Languages — C++ (ISO/IEC 14882:2017). Final Committee Draft N4659. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf